Lab 6: Code and Branch Coverage Analysis

Software Testing · Application under test: `pet_boarding.py`

NAME
Madhusudan P

SRN
PES1UG24AM375

GROUP
L6_L7_49

TOOLS
Python 3.12, pytest, coverage.py 7.17

01 Objective

- Differentiate line (statement) coverage from branch coverage.
- Use `coverage.py` to measure coverage and read the report, including the *Missing* column.
- Write tests that cover every statement and both outcomes of every decision, including boundary values.

02 Summary of Results

100%	100%	37/37	26/26	23
Line coverage	Branch coverage	Statements covered	Branches covered	Tests (all passed)

Both results exceed the 90% requirement of the lab.

03 Application Under Test

`pet_boarding.py` has three functions and a lookup table (`INCLUDED_DAYS`):

Function	Purpose	Decision points
`boarding_fee(pet_type, days, is_premium)`	Computes the boarding fee. Free days depend on pet type; premium adds one free day (not for rabbits). Extra days are charged at 30 (dog) or 20 (others), with a surcharge when extra days exceed 10.	7 decisions, 14 branches

`admission_ok(pet_age_years, vaccinations, is_senior)`	Seniors need at least 3 vaccinations; others must be at least 1 year old with at least 2 vaccinations.	2 decisions, 4 branches
`checkout_status(minutes_left, is_long_stay, is_weekend)`	Returns open, late or closed from a cutoff (15 or 30 minutes, plus 20 on weekends).	4 decisions, 8 branches

Total: **26 branches** across **37 statements**.

04 Line (Code) Coverage

Line coverage asks whether each statement ran at least once.

```
$ coverage run -m pytest test_pet_boarding.py -v
```

```
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-9.2.0, pluggy-1.7.0 -- /usr/bin/python3
rootdir: /home/claude/lab
collecting ... collected 23 items

test_pet_boarding.py::test_within_included_days PASSED                   [  4%]
test_pet_boarding.py::test_unknown_pet_type_raises PASSED                [  8%]
test_pet_boarding.py::test_negative_days_raises PASSED                   [ 13%]
test_pet_boarding.py::test_premium_adds_a_day_for_dog PASSED             [ 17%]
test_pet_boarding.py::test_premium_ignored_for_rabbit PASSED             [ 21%]
test_pet_boarding.py::test_dog_extra_days_rate_30 PASSED                 [ 26%]
test_pet_boarding.py::test_cat_extra_days_rate_20 PASSED                 [ 30%]
test_pet_boarding.py::test_dog_long_stay_surcharge PASSED                [ 34%]
test_pet_boarding.py::test_cat_long_stay_surcharge_no_dog_bonus PASSED   [ 39%]
test_pet_boarding.py::test_rabbit_extra_days_rate_20 PASSED              [ 43%]
test_pet_boarding.py::test_extra_exactly_zero_boundary PASSED            [ 47%]
test_pet_boarding.py::test_admission_ok PASSED                           [ 52%]
test_pet_boarding.py::test_admission_senior_enough_vaccinations PASSED   [ 56%]
test_pet_boarding.py::test_admission_senior_too_few_vaccinations PASSED  [ 60%]
test_pet_boarding.py::test_admission_young_pet_rejected PASSED           [ 65%]
test_pet_boarding.py::test_admission_adult_too_few_vaccinations PASSED   [ 69%]
test_pet_boarding.py::test_checkout_open_short_weekday PASSED            [ 73%]
test_pet_boarding.py::test_checkout_late_short_weekday PASSED            [ 78%]
test_pet_boarding.py::test_checkout_closed_short_weekday PASSED          [ 82%]
test_pet_boarding.py::test_checkout_long_stay_cutoff PASSED              [ 86%]
test_pet_boarding.py::test_checkout_weekend_raises_cutoff PASSED         [ 91%]
test_pet_boarding.py::test_checkout_long_stay_weekend_open PASSED        [ 95%]
test_pet_boarding.py::test_checkout_boundary_open_exact PASSED           [100%]

============================== 23 passed in 0.03s ==============================
```

```
$ coverage report -m
```

```
Name                  Stmts   Miss  Cover   Missing
---------------------------------------------------
pet_boarding.py          37      0   100%
test_pet_boarding.py     50      0   100%
---------------------------------------------------
TOTAL                    87      0   100%
```

Result 37 statements, 0 missed, **100%** line coverage. The *Missing* column is empty.

05 Branch Coverage

Branch coverage asks whether every decision evaluated to both True and False. It is stricter than line coverage. For example, a test that reaches the `return` after an `if` can cover the line without ever testing the case where the `if` is False.

```
$ coverage run --branch -m pytest test_pet_boarding.py
```

```
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-9.2.0, pluggy-1.7.0
rootdir: /home/claude/lab
plugins: platformdirs-4.13.0
collected 23 items

test_pet_boarding.py .......................                             [100%]

============================== 23 passed in 0.03s ==============================
```

```
$ coverage report -m
```

```
Name                  Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------------------
pet_boarding.py          37      0     26      0   100%
test_pet_boarding.py     50      0      0      0   100%
-----------------------------------------------------------------
TOTAL                    87      0     26      0   100%
```

Result 26 branches, 0 partial (`BrPart` = 0), **100%** branch coverage.

Branch coverage map

Function	Decision	True outcome covered by	False outcome covered by
boarding_fee	`pet_type not in INCLUDED_DAYS`	test_unknown_pet_type_raises	test_within_included_days
boarding_fee	`days < 0`	test_negative_days_raises	test_within_included_days
boarding_fee	`is_premium and pet_type != "rabbit"`	test_premium_adds_a_day_for_dog	test_premium_ignored_for_rabbit
boarding_fee	`extra <= 0`	test_within_included_days, test_extra_exactly_zero_boundary	test_dog_extra_days_rate_30
boarding_fee	`pet_type == "dog" (rate)`	test_dog_extra_days_rate_30	test_cat_extra_days_rate_20

boarding_fee	`extra > 10`	test_dog_long_stay_surcharge	test_dog_extra_days_rate_30
boarding_fee	`pet_type == "dog"` (surcharge)	test_dog_long_stay_surcharge	test_cat_long_stay_surcharge_no_dog_bonus
admission_ok	`is_senior`	test_admission_senior_enough_vaccinations	test_admission_ok
admission_ok	`pet_age_years < 1`	test_admission_young_pet_rejected	test_admission_ok
checkout_status	`is_long_stay`	test_checkout_long_stay_cutoff	test_checkout_open_short_weekday
checkout_status	`is_weekend`	test_checkout_weekend_raises_cutoff	test_checkout_open_short_weekday
checkout_status	`minutes_left >= cutoff`	test_checkout_open_short_weekday	test_checkout_late_short_weekday
checkout_status	`minutes_left >= cutoff - 15`	test_checkout_late_short_weekday	test_checkout_closed_short_weekday

06 Test Case Design

Techniques used: **decision coverage** (one test per outcome), **boundary value analysis** (`extra == 0`, `minutes_left == cutoff`), **error guessing** (invalid pet type, negative days) and **condition isolation** for the compound condition `is_premium and pet_type != "rabbit"`.

#	Function	Test	Input	Expected	Covers
1	boarding_fee	`test_within_included_days`	`dog, 1, False`	`0`	extra ≤ 0
2	boarding_fee	`test_unknown_pet_type_raises`	`hamster, 5, False`	`ValueError`	invalid pet type
3	boarding_fee	`test_negative_days_raises`	`cat, -1, False`	`ValueError`	days < 0
4	boarding_fee	`test_premium_adds_a_day_for_dog`	`dog, 3, True`	`0`	premium AND not rabbit = True
5	boarding_fee	`test_premium_ignored_for_rabbit`	`rabbit, 5, True`	`20`	premium AND not rabbit = False
6	boarding_fee	`test_dog_extra_days_rate_30`	`dog, 5, False`	`90`	dog rate 30
7	boarding_fee	`test_cat_extra_days_rate_20`	`cat, 6, False`	`60`	non-dog rate 20

8	boarding_fee	`test_dog_long_stay_surcharge`	`dog, 15, False`	`585`	extra > 10, dog +15
9	boarding_fee	`test_cat_long_stay_surcharge_no_dog_bonus`	`cat, 15, False`	`360`	extra > 10, non-dog +10
10	boarding_fee	`test_rabbit_extra_days_rate_20`	`rabbit, 6, False`	`40`	rabbit rate 20
11	boarding_fee	`test_extra_exactly_zero_boundary`	`cat, 3, False`	`0`	boundary extra = 0
12	admission_ok	`test_admission_ok`	`4, 2, False`	`True`	adult, enough vaccinations
13	admission_ok	`test_admission_senior_enough_vaccinations`	`9, 3, True`	`True`	senior, ≥ 3 vaccinations
14	admission_ok	`test_admission_senior_too_few_vaccinations`	`9, 2, True`	`False`	senior, < 3 vaccinations
15	admission_ok	`test_admission_young_pet_rejected`	`0, 5, False`	`False`	age < 1
16	admission_ok	`test_admission_adult_too_few_vaccinations`	`3, 1, False`	`False`	adult, < 2 vaccinations
17	checkout_status	`test_checkout_open_short_weekday`	`20, False, False`	`open`	cutoff 15, 20 ≥ 15
18	checkout_status	`test_checkout_late_short_weekday`	`5, False, False`	`late`	0 ≤ 5 < 15
19	checkout_status	`test_checkout_closed_short_weekday`	`-1, False, False`	`closed`	-1 < 0
20	checkout_status	`test_checkout_long_stay_cutoff`	`29, True, False`	`late`	cutoff 30
21	checkout_status	`test_checkout_weekend_raises_cutoff`	`34, False, True`	`late`	cutoff 15+20 = 35
22	checkout_status	`test_checkout_long_stay_weekend_open`	`50, True, True`	`open`	cutoff 30+20 = 50
23	checkout_status	`test_checkout_boundary_open_exact`	`15, False, False`	`open`	boundary minutes = cutoff

Expected values come from the arithmetic in each test's comments and were confirmed by running the suite (all 23 pass). Example: `boarding_fee("dog", 15, False)` gives included = 2, extra = 13, rate = 30 + 10 + 5 = 45, fee = 13 × 45 = **585**.

07 Line vs Branch Coverage Observations

- Line coverage only confirms each statement executed. It cannot tell if an `if` was never False.
- In `boarding_fee`, `pet_type == "dog"` appears twice (rate and surcharge). Both need a dog and a non-dog case, which is why separate dog and cat long-stay tests exist.
- The compound condition `is_premium and pet_type != "rabbit"` needs both a premium non-rabbit (adds a day) and a premium rabbit (does not).
- Boundary tests (`extra == 0`, `minutes_left == cutoff`) catch off-by-one errors in `<=` and `>=` that coverage alone would not flag.
- 100% coverage shows every path was executed, not that the logic is correct. The assertions on exact values are what check correctness.

08 Commands to Reproduce

```
pip install coverage pytest
coverage run -m pytest test_pet_boarding.py
coverage report -m
coverage erase
coverage run --branch -m pytest test_pet_boarding.py
coverage report -m
```

09 Deliverables Checklist

Deliverable	Status
Line coverage terminal output	Included (section 04)
Branch coverage terminal output	Included (section 05)
`CodeCoverageTest_pet_boarding.py`	Commit `test_pet_boarding.py` (line coverage ≥ 90%) under this name
`BranchCoverageTest_pet_boarding.py`	Commit the same file (branch coverage ≥ 90%) under this name
Minimum 2 commits	One commit per file above, with a screenshot of the matching terminal output

10 Conclusion

The test suite of 23 tests achieves **100% line coverage (37/37 statements)** and **100% branch coverage (26/26 branches)** on `pet_boarding.py`, with all tests passing. This meets and exceeds the 90% target for both metrics.